

Life RPG — Action Plan for Round 1 Submission

Tech Zephyr 4.0 · IIT Bhubaneswar · Prepared for team DayOne

Current Status Snapshot

Area	Status	Est. Done
UI / theme (dashboard, shop, tasks, boss fight)	Built and functional	~55%
Non-linear XP leveling formula	Implemented correctly	Done
Task create / complete / delete	Working	Done
Streak system	Hardcoded to 1, never updates	~10%
Real user auth	Email textbox only, no password	0%
Backend + database	None — pure localStorage	0%
Accessibility (aria / alt / keyboard)	None found in code	0%
SEO meta tags	Only a <title> tag	~10%
README accuracy	Claims React/Next.js/Supabase — not actually	My smatch

Overall: roughly 35–40% of the full checklist is genuinely done. The critical gap is the missing backend/database/auth — this is exactly what the problem statement's zero-tolerance rules ("Fake Data Persistence", "Invalid Repository") test for.

Note: certificate policy for non-qualifying / flagged submissions was not stated in the rulebook — worth a quick check on the Unstop event page or in the WhatsApp group.

Action Plan — Do These In Order

Steps 1–3 are the ones that protect you from automatic disqualification. If time runs out, finish those first and submit — a plain-but-working app beats a pretty app that gets auto-zeroed.

1. Set up Supabase (free tier)

Create a free Supabase project. This gives you a real Postgres database plus built-in email/password authentication in one place, with almost no server code needed — it's explicitly allowed in the rules as backend-as-a-service. Grab your Project URL and anon key from the project settings.

CRITICAL — prevents automatic disqualification

2. Replace fake login with real Supabase Auth

Swap the current "type any email, click login" box for `supabase.auth.signUp()` and `signInWithPassword()`. This alone fixes the biggest disqualification risk (no real auth) and takes roughly 30–45 minutes using Supabase's JS SDK.

CRITICAL — prevents automatic disqualification

3. Replace localStorage with Supabase database calls

Create a 'user_data' table (level, xp, gold, str, int, streak, inventory) and a 'tasks' table, both linked to the logged-in user's id. Swap saveData() / loadUserData() to insert/select/update from Supabase instead of localStorage. This removes the "Fake Data Persistence" zero-tolerance risk.

CRITICAL — prevents automatic disqualification

4. Make the streak system real

Store a 'last_active_date' field. On login, compare it to today: if it was yesterday, increment the streak; if it's today, leave it unchanged; if older than yesterday, reset to 1. Right now it is hardcoded to "1" and never changes.

5. Add basic accessibility

Add aria-label to icon-only buttons, alt text to any images/emoji-icons that convey meaning, and confirm every interactive element is reachable and operable with Tab + Enter/Space. Native tags already get keyboard support for free — just make sure nothing is a plain .

6. Add SEO meta tags

In the , add a tag and basic Open Graph tags (og:title, og:description). Takes about 5 minutes and is a direct judging pillar most teams skip entirely.

7. Fix the README to match reality

The README currently claims React/Next.js/Supabase were used when the code is vanilla HTML/JS with localStorage. Once Supabase is actually integrated (steps 1–3), update the README so it reflects exactly what's true. A mismatched README next to reviewed commit history reads as a red flag.

8. Add .env.example, verify commits, record the video

Add a .env.example with placeholder Supabase URL/key names. Double-check GitHub commits are timestamped after 12 Sept 2026, 10:00 AM — the rulebook explicitly says commit history will be reviewed. Record a 90–180 second video: signup -> add a quest -> complete it -> level up -> refresh the page to prove the data persisted in the database, not localStorage.

Reminder — Zero-Tolerance Rules (auto-zero if triggered)

- Broken Links — GitHub repo private, or live link broken at judging time
- Fake Data Persistence — app relies solely on localStorage for primary data
- Build/Deployment Failure — app crashes on load, or backend fails to connect to the database in production
- Console/Runtime Crashes — unhandled errors or blank-screen crashes during basic usage
- Invalid Repository — fewer than 3 chronological commits, missing backend code, or signs of a single dump commit
- Missing/Restricted Video — walkthrough video omitted, requires login to view, or exceeds 100MB